INTERNSHIP PROJECT SPECIFICATION

ServiceHub

Enterprise Service Desk & Maintenance Management System

Project type: Full-stack enterprise web application
Duration: 3–4 weeks | **Team size:** 3–4 interns
Required stack: React, Redux Toolkit, Spring Boot, Spring Security, PostgreSQL, Docker

A practical assignment focused on real business workflows, secure API development, relational data modeling, frontend state management, testing, and containerized delivery.

Prepared as a student-ready project brief. Teams should use this document as the baseline specification and document any approved design decisions.

1. Project Overview

ServiceHub is an internal service management platform for an organization with multiple departments and employees. Employees submit technical support or maintenance requests; technicians work on assigned requests; managers review department activity; and administrators manage users and system configuration.

The platform replaces scattered requests made through phone calls, emails, or verbal communication with a centralized, traceable workflow.

Example business scenarios

- An employee's computer stops working and they submit a hardware support request.
- An office printer needs maintenance and the request is assigned to a technician.
- An employee requests installation of approved software.
- A department requests a new computer or network support.
- An office reports an electrical or facility maintenance issue.

Project objectives

- Build a realistic full-stack application using a React frontend and Spring Boot REST API.
- Implement authentication and role-based authorization on the backend.
- Design and implement a relational PostgreSQL database.
- Use frontend state management appropriately, including local and global state.
- Run the complete application using Docker Compose.
- Produce professional technical documentation and automated tests.

Scope note: Build this first version as a modular monolith. Microservices, message brokers, and distributed infrastructure are intentionally out of scope for the core assignment.

2. User Roles and Permissions

Role	Responsibilities
ADMIN	Manage users, departments, service categories, all requests, and system configuration.
EMPLOYEE	Create requests, view own requests, and add comments to own requests.
TECHNICIAN	View assigned requests, update permitted statuses, and add work notes.
MANAGER	View department requests, monitor performance, assign or approve requests where permitted.

Permission matrix

Feature	Admin	Employee	Technician	Manager
Create request	Yes	Yes	Yes	Yes
View all requests	Yes	No	No	Department
View own requests	Yes	Yes	Yes	Yes
Assign technician	Yes	No	No	Yes
Update request status	Yes	No	Yes*	No
Approve requests	Yes	No	No	Yes
Manage users	Yes	No	No	No
View reports	Yes	No	Assigned work	Department

*Technicians may update only requests assigned to them, and only through valid workflow transitions. Managers are limited to their department unless explicitly granted broader access.

Authorization must be enforced by Spring Boot. Hiding a button in React is not a security control.

3. Functional Requirements

3.1 Authentication and User Management

- Provide login and logout.
- Use BCrypt to hash passwords; never store plaintext passwords.
- Issue and validate JWT access tokens.
- Support administrator-controlled user creation or invitation-based registration.
- Allow authorized users to view and update profile information.
- Allow administrators to activate and deactivate accounts.
- Assign users to departments and roles.
- Prevent users from changing their own roles or privilege level.

Inactive users must not be able to authenticate. Protected API endpoints must reject missing, invalid, or expired tokens.

3.2 Service Request Management

Employees and other permitted users can create service requests with the following information:

- Request title and detailed description.
- Service category, such as hardware, software, network, or maintenance.
- Priority: LOW, MEDIUM, HIGH, or CRITICAL.
- Department and location.
- Optional attachments as a stretch goal.
- A unique reference number, for example SR-2026-0001.

Required request lifecycle

Status	Meaning
SUBMITTED	A user has created a new request.
PENDING	The request is awaiting review or assignment.
ASSIGNED	A technician has been assigned.
IN_PROGRESS	The technician is actively working on the request.
RESOLVED	The reported issue has been addressed.
CLOSED	The request has been confirmed and closed.

The system must record every status change, including the previous status, new status, actor, timestamp, and optional note. Invalid transitions must be rejected by the backend.

3.3 Technician Assignment

- Authorized administrators and managers can view available technicians.
- Assign a request to a technician and reassign it when necessary.
- View technician workloads and filter requests by assignee.
- Technicians can view and manage requests assigned to them.
- The backend must verify assignment ownership before allowing technician actions.

3.4 Comments and Communication

- Users can add comments to requests they are authorized to access.
- Technicians can add work notes.
- Each comment displays its author and creation timestamp.
- Support internal notes that are visible only to authorized staff.
- The API must prevent employees from reading internal notes.

3.5 Dashboard and Reporting

- Show total, pending, in-progress, resolved, and critical requests.
- Show request counts by category and priority.
- Show workload by technician.
- Calculate average resolution time.
- Support filtering by date range, department, priority, and status.
- All dashboard metrics must come from backend APIs and real database data.

4. Technical Requirements

Layer	Required technologies / tools
Frontend	React, TypeScript, React Router, Redux Toolkit, Axios, React Hook Form, Zod
Backend	Java, Spring Boot, Spring Web, Spring Data JPA, Spring Security, JWT
Database	PostgreSQL
Migrations	Flyway
API documentation	Swagger / OpenAPI
Testing	JUnit 5, Mockito; frontend tests using an appropriate React testing tool
Infrastructure	Docker, Docker Compose, Maven, Node.js

Recommended backend additions

- Spring Validation for request validation.
- MapStruct for DTO/entity mapping (recommended).
- Global exception handling using @ControllerAdvice.
- Consistent response and error formats.
- Pagination, sorting, filtering, and search.
- Use environment variables for secrets and environment-specific configuration.

Frontend state management expectations

State / concern	Recommended approach
Modal visibility, dropdown state	React useState
Form inputs and validation	React Hook Form + Zod
Authenticated user and role	Redux Toolkit authentication slice
Global UI filters	Redux Toolkit when shared across pages
API data and caching	RTK Query or another documented server-state solution

Do not put every state variable in Redux. Students should explain why a piece of state is local, global, or server-owned.

5. Suggested Application Architecture

Use a layered Spring Boot architecture. Controllers handle HTTP concerns; services enforce business rules; repositories access persistence; DTOs define API contracts; and security configuration protects endpoints.

Backend package	Responsibility
config/	Application configuration, CORS, OpenAPI, and related setup.
security/	JWT processing, authentication, authorization, and security filters.
controller/	REST endpoints and HTTP request/response handling.
service/	Business logic and workflow validation.
repository/	Spring Data JPA persistence operations.

Backend package	Responsibility
entity/	JPA entities and relationships.
dto/	Request and response objects.
mapper/	DTO/entity mapping.
exception/	Custom exceptions and global error handling.

Suggested frontend structure

Organize the React application by feature so related pages, components, API calls, types, and state remain close together.

```
frontend/ src/ app/ store.ts, hooks.ts components/ shared UI components features/ auth/ requests/ users/ dashboard/ pages/ services/ API client and shared API setup routes/ route definitions and guards hooks/ types/ utils/ Dockerfile package.json
```

Suggested backend structure

```
backend/ src/main/java/com/servicehub/ config/ security/ controller/ service/ repository/ entity/ dto/ mapper/ exception/ ServiceHubApplication.java
```

6. Database Design Requirements

Students must design the relational database and submit an ER diagram before implementing the persistence layer. The following entities are mandatory:

Entity / table	Purpose
users	User accounts, password hashes, department and role references.
roles	Role definitions: ADMIN, EMPLOYEE, TECHNICIAN, MANAGER.
departments	Organization departments.
service_categories	Categories such as hardware, software, network, and maintenance.
service_requests	Request details, priority, status, requester, department, and assignment.
request_comments	Public discussion comments and internal work notes.
request_status_history	Status transitions, actor, timestamp, and optional note.

Required relationships

- One department has many users.
- One role can be assigned to many users.
- One user can create many service requests.
- One technician can receive many service request assignments.
- One service request can have many comments.
- One service request can have many status history records.

Database implementation requirements

- Use primary keys and foreign keys appropriately.
- Define one-to-many and other necessary relationships.
- Use unique constraints for email and request reference number.
- Add indexes for frequently filtered columns where appropriate.
- Use created_at and updated_at timestamps.
- Preserve referential integrity.
- Use Flyway migrations rather than relying only on automatic schema generation.
- Avoid storing derived values that can be calculated reliably, unless there is a documented reason.

7. REST API Requirements

The backend must expose documented REST endpoints. The following list is a baseline; teams may add endpoints when justified by the design.

Method	Endpoint	Purpose
POST	/api/auth/login	Authenticate a user.
POST	/api/auth/register	Register a user, if registration is enabled.
GET	/api/users/me	Get the authenticated user's profile.
GET	/api/users	List users for authorized administrators.

Method	Endpoint	Purpose
POST	/api/requests	Create a service request.
GET	/api/requests	List requests visible to the current user.
GET	/api/requests/{id}	Get request details.
PUT	/api/requests/{id}	Update permitted request fields.
PATCH	/api/requests/{id}/assign	Assign or reassign a technician.
PATCH	/api/requests/{id}/status	Change request status.
POST	/api/requests/{id}/comments	Add a comment or work note.
GET	/api/requests/{id}/comments	List comments visible to the user.
GET	/api/dashboard/summary	Get dashboard statistics.

Example filtered request query

```
GET /api/requests?page=0&size=10&status=PENDING&priority=HIGH
```

API standards

- Use pagination, sorting, search, and filtering where appropriate.
- Use correct HTTP status codes, including 200, 201, 204, 400, 401, 403, 404, and 409 where applicable.
- Validate all client-supplied input on the backend.
- Return consistent response and error structures.
- Document endpoints, request schemas, response schemas, and security requirements in Swagger/OpenAPI.
- Never trust a user ID, role, department, or assignment supplied by the client without authorization checks.

Data isolation rule: the backend must derive the authenticated user from the security context and enforce record-level access. It must not return another employee's private requests simply because the frontend asks for them.

8. Authentication and Security Requirements

Mandatory security requirements

1. Authenticate users using email and password.
2. Hash passwords using BCrypt.
3. Generate and validate JWT access tokens.
4. Protect authenticated endpoints using Spring Security.
5. Implement role-based access control.
6. Restrict users to records and actions they are authorized to access.
7. Validate request data and reject invalid workflow transitions.
8. Prevent unauthorized role or privilege changes.
9. Configure CORS for the expected frontend origin.
10. Return appropriate 401 Unauthorized and 403 Forbidden responses.

Required authorization test scenarios

- Employee A cannot access Employee B's private requests.
- An employee cannot assign a technician.
- A technician cannot modify a request assigned to another technician.
- An unauthenticated user cannot access protected endpoints.
- A manager cannot manage system-wide users unless explicitly granted that permission.
- An employee cannot view internal technician notes.
- A deactivated user cannot continue using the application.

For this training project, JWT access tokens are sufficient. If refresh tokens are implemented, use secure storage, expiration, and rotation. Do not casually store long-lived credentials in browser local storage.

9. Docker and Local Execution

The complete application must run using Docker Compose. At minimum, provide containers for PostgreSQL, the Spring Boot backend, and the React frontend.

Illustrative compose.yaml starting point

```
services: postgres: image: postgres:17 environment: POSTGRES_DB: servicehub POSTGRES_USER: servicehub POSTGRES_PASSWORD: ${DB_PASSWORD} ports: - "5432:5432" volumes: - postgres_data:/var/lib/postgresql/data backend: build: ./backend depends_on: - postgres ports: - "8080:8080" frontend: build: ./frontend ports: - "3000:80" depends_on: - backend volumes: postgres_data:
```

This is an illustrative starting point, not a complete production Compose file. Teams must add health checks, environment configuration, service readiness handling, and appropriate network configuration. The backend should connect to the database using the Compose service hostname (postgres), not localhost.

Start the application

```
docker compose up --build
```

Component	Expected local URL
Frontend	http://localhost:3000

Component	Expected local URL
Backend	http://localhost:8080
Swagger UI	http://localhost:8080/swagger-ui/index.html

Do not commit real database passwords, JWT secrets, or other credentials. Provide a .env.example file with placeholder values and document the required environment variables.

10. Development Milestones

Period	Work	Deliverable
Week 1 – Analysis and foundation	Understand requirements; prepare SRS and use cases; design ERD; create React and Spring Boot projects; configure PostgreSQL and Docker; implement initial entities.	Approved architecture, ERD, project skeleton, and working Docker environment.
Week 2 – Backend	Implement authentication and authorization; build user and department APIs; implement request CRUD, assignment, validation, and exception handling.	Functional REST API with Swagger documentation.
Week 3 – Frontend	Build login and dashboard; configure Redux Toolkit; implement protected routes; build request, comment, and assignment interfaces; integrate APIs.	Functional React application integrated with backend.
Week 4 – Integration and delivery	Complete dashboard analytics; write tests; verify authorization scenarios; finalize Docker Compose; fix bugs; prepare documentation and demo.	Working containerized application, tests, documentation, and presentation.

11. Required Documentation

Document	Required contents
SRS	Project scope, stakeholders, functional requirements, non-functional requirements, assumptions, and constraints.
Use Case Diagram	Actors, system boundary, and user interactions.
ERD	Tables/entities, attributes, keys, and relationships.
Architecture Diagram	Frontend, backend layers, database, and communication paths.
API Documentation	Swagger/OpenAPI endpoints, request/response examples, and security requirements.
README	Overview, stack, prerequisites, environment variables, setup, Docker commands, demo accounts, and screenshots.
Test Report	Test scenarios, expected results, actual results, and defects found.
User Manual	Instructions for each role and common workflows.

README checklist

- Project overview and feature list.
- Technology stack and architecture summary.
- Prerequisites and installation instructions.
- Environment variables and .env.example.
- Docker Compose commands.
- Demo accounts and their roles (use non-production credentials).
- Swagger documentation URL.
- Screenshots and known limitations.

12. Evaluation Criteria

Assessment area	Weight
Backend architecture and REST APIs	20%
Database design and relationships	15%
Authentication and authorization	20%
React and state management	15%
Docker and deployment	10%
Testing and code quality	10%
Documentation and presentation	10%
TOTAL	100%

Assessment should focus on engineering quality, correctness, security, maintainability, and the intern's ability to explain their implementation—not merely on visual appearance.

13. Bonus Challenges

These features are optional. Teams should complete the core requirements before attempting bonus work.

- Email notifications when a technician is assigned.
- File attachments for service requests.
- Request escalation for overdue tasks.
- Audit logs for important administrative actions.
- Export reports to CSV or PDF.
- Real-time notifications using WebSockets.
- Service-level agreement (SLA) monitoring.
- Advanced dashboard analytics.

14. Final Submission Requirements

Each team must submit all of the following:

1. GitHub repository with a clean and understandable commit history.
2. Functional React frontend.
3. Functional Spring Boot backend.
4. PostgreSQL database with versioned migrations.
5. Docker Compose configuration that starts the complete application.
6. Swagger/OpenAPI documentation.
7. SRS, ERD, use-case diagram, and architecture diagram.
8. Automated tests and a test report.
9. Working demonstration of the main workflows.
10. Individual contribution report from every team member.

During the final presentation, any team member may be asked to explain any part of the system. Each intern is expected to understand the overall application, even when tasks are divided among team members.

15. Mentor Guidance and Recommended Delivery Approach

Recommended team size: 3–4 interns. Teams may divide initial responsibilities across frontend, backend, database, and integration, but every intern should complete at least one end-to-end feature spanning the UI, API, business logic, and database.

Checkpoint	Mentor review
Checkpoint 1 – Before implementation	Team explains the business requirements, ERD, architecture, authentication approach, and key authorization rules.
Checkpoint 2 – API and integration	Team demonstrates working APIs, database persistence, JWT protection, and at least one integrated React workflow.
Checkpoint 3 – Final demonstration	Team demonstrates role-specific workflows, security tests, Docker startup, documentation, and explains design decisions.

Mentor recommendation

Keep this project as a modular monolith. It provides meaningful enterprise workflows while keeping the infrastructure manageable for new interns. After the team demonstrates sound application architecture, database design, and security fundamentals, a later assignment can introduce microservices, RabbitMQ, Redis, and event-driven architecture.

Definition of Done

- A fresh clone can be started using the documented Docker Compose instructions.
- The database schema is created through migrations.
- The core workflows work for all four roles.
- Backend authorization tests cover record-level access and role restrictions.
- The frontend handles loading, error, empty, and success states.
- The API is documented and the README is complete.
- The team can explain the architecture and demonstrate the application without relying on undocumented manual setup.

End of project specification